

AP157 Output 8 - March 27, 2026

Benedict A. Pangilinan (bapangilinan5@up.edu.ph)

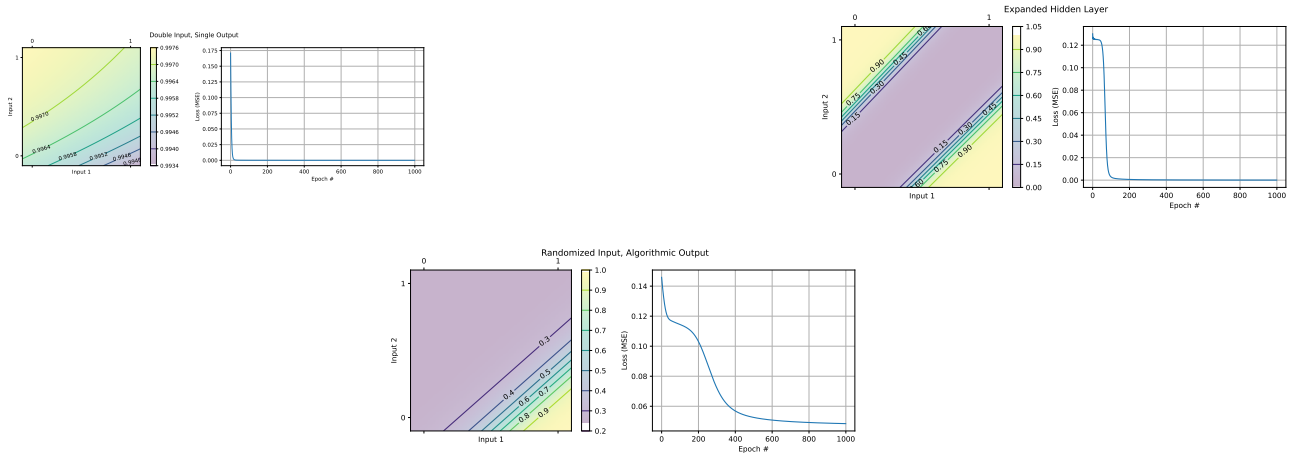


Figure 1: **Different Schemes.** For XOR functions, we should see output of 1 restricted near $[0,0]$ and $[1,1]$, where the first and second coordinates are Inputs 1 and 2, respectively. (*Top Left*) The output when input is reduced to two occurrences and one output, (*Top Right*) When there are multiple hidden layers instead of one, (*Bottom*) When we have the standard four inputs and four outputs but the former is randomized and the latter is algorithmic.

Double Input, Single Output

Here, the standard code is modified by replacing the `xor_in` and `xor_out` with:

```
xor_in = torch.tensor([[0,0],[0,1]], dtype=torch.float32)
xor_out = torch.tensor([1], dtype=torch.float32).unsqueeze(1)
```

Given `torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)` gives rise to four outputs 0, 1, 1, and 0, respectively. Choosing randomly, they do not give the correct output. As we can see, the model practically gives 1 in the entire map. Iterations of running might change the values but still consistently above 0.99. This effectively teaches the nontriviality of missing features, and why neural networks are just generalized regression.

Expanded Hidden Layers

Here, we do the following modification from just a single hidden layer. It gets the right results, but is converging to output 1 at smaller values of both Inputs 1 and 2, as you can see in Fig. . The error plot also crashed quickly to 0, suggestive of exaggerated convergence, and perhaps overfitting. We are merely dealing with XOR functions of binary input and output here, but one can imagine how it would react when dealing with continuous nonlinear functions. At smaller learning rates, XOR function is also unable to come like in other cases discussed here. (Not shown here)

Randomized Input

As you can notice, we sum the values in the binarized sub-array (the four arrays forming our input array) and take the modulus of 2 to generalize what we have in the standard. As you can manually verify, same elements give 0 and different elements give 1 under that algorithm:

```
# Number of samples
N = 8 # We divide them into four input arrays

# Randomized inputs between 0 and 1 using NumPy
xor_in = np.random.rand(N, 2) # shape (N,2),
    values in [0,1]

# Binarize inputs via threshold at 0.5
bin_xor_in = (xor_in >= 0.5).astype(np.float32)
# Define XOR output: sum each row and take modulus
2
raw_xor_out = (bin_xor_in.sum(axis=1) % 2).astype(
    np.float32).reshape(-1,1)

# Convert to torch tensors if needed
xor_in_torch = torch.tensor(bin_xor_in, dtype=
    torch.float32)
xor_out = torch.tensor(raw_xor_out, dtype=torch.
    float32)
```

Bottom subplot is shown since it is the failure case, but you can have cases where it gets the standard result. The mistakes stem from uniformity of outputs when an iteration does not span all possible double-element sub-arrays.